

TICKET BOOKING MANAGEMENT

School of Computing

**Bachelor of Technology
in
Information Technology**

By

Mohamed Shameel Imran M (VTU26959)

10211IT224

Full Stack Application Development

Slot : S2L5L7



**DEPARTMENT OF INFORMATION TECHNOLOGY
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr. SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "TICKET BOOKING MANAGEMENT" by Mohamed Shameel Imran M (VTU26959) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Vismathi J

Professor

Information Technology

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Head of the Department

Dr. A. Peter Soosai Anandaraj

Professor & Head

Information Technology

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Mohamed Shameel Imran M

Date: 06/04/2026

ABSTRACT

EventSphere is a full-stack event management and ticketing platform designed to improve event discovery and booking within campus environments. The system enables users to browse categorized events, securely reserve tickets, and receive QR-coded confirmations for digital verification via email. It is developed using React 19 with Vite 8 on the frontend and Java 17 with Spring Boot 3.2 on the backend, supported by a MySQL database. A real-time concurrency control mechanism is implemented to prevent double-booking during simultaneous reservations, ensuring data consistency and reliability. The platform also incorporates role-based access control, allowing administrators to manage events and tickets efficiently, along with a rule-based smart assistant. Overall, EventSphere demonstrates the integration of modern web technologies, secure booking mechanisms, and intelligent user interaction in a scalable and user-friendly application.

Keywords: Event Management, Ticket Booking, Spring Boot, React, QR Code, Concurrency Control, ConcurrentHashMap, ZXing, Asynchronous Email, MySQL, JPA, Role-Based Access Control, Full Stack Web Application

LIST OF FIGURES

1.1	Propose System Architecture	6
3.1	Data Flow Diagram	13
5.1	EventSphere Landing Page Interface	27
5.2	EventSphere Event Browsing Screen with Smart As- sistant	28
5.3	EventSphere Real-Time Ticket Locking Interface with Countdown Timer	29
5.4	EventSphere Booking Confirmation Screen with QR Code Verification	30

LIST OF TABLES

5.1	Feature Comparison: EventSphere vs Similar Platforms	26
7.1	Mapping of Project to Complex Engineering Problem (CEP)	35
7.2	Mapping of Project to Complex Engineering Activities (CEA)	36
7.3	SDG Mapping for the Project	37

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
DB	Database
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model-View-Controller
MySQL	My Structured Query Language
ORM	Object Relational Mapping
PNG	Portable Network Graphics

QR	Quick Response
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.4 Framework	4
2 SURVEY OF SIMILAR APPLICATION IN MARKET	7
3 FRAMEWORK DESCRIPTION	9
3.1 Frontend Implementation	9

3.1.1	Environment Setup	9
3.1.2	Structure of the Project	10
3.1.3	Execution Procedure	11
3.2	Backend Implementation	11
3.2.1	Environment Setup	11
3.2.2	Structure of the Project	12
3.2.3	Execution Procedure	12
3.3	Database Implementation	14
3.3.1	Database Configuration	14
3.3.2	Schema Structure	14
3.3.3	Tables Created	15
4	METHODOLOGIES	16
4.1	Project Architecture	16
4.2	DataFlow Diagram	17
4.3	Module 1: User Authentication and Authorization Module	17
4.4	Module 2: Event Discovery and Smart Assistant Module	18
4.5	Module 3: Ticket Booking, QR Generation, and Email Notification Module	19
5	RESULTS AND DISCUSSIONS	24

6	CONCLUSION AND FUTURE ENHANCEMENTS	31
6.1	Conclusion	31
6.2	Future Enhancements	32
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	34
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	34
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	36
7.3	SDG Mapping	37
	References	37

Chapter 1

INTRODUCTION

1.1 Introduction

EventSphere is a comprehensive, full-stack Event Management and Ticketing platform experience as identified by Liu et al. (2023) [6] developed to address the challenges associated with organizing and attending campus-level departmental events. In a college environment, events such as hackathons, workshops, technical seminars, and non-technical competitions are frequently conducted across multiple departments. However, the lack of a centralized, digital platform often results in manual registration processes, uncontrolled crowd management, and the risk of double-booking — all of which degrade both the organizer’s and participant’s experience.

EventSphere solves these problems by providing a unified digital platform where students can explore events filtered by their department and category, book tickets securely with real-time availability guaran-

tees as explored in digital ticketing systems by Ferreira et al. (2023) [2] and receive a digitally verifiable QR-coded ticket directly to their email. The platform is designed around a premium, dark-themed user interface featuring glassmorphism effects and smooth micro-animations, ensuring an engaging and professional user experience.

The system is developed as a two-tier application consisting of a React-based Single Page Application (SPA) on the frontend and a RESTful API server built with Spring Boot, following best practices described by Nair et al. (2023) [8] on the backend. Data persistence is handled by a MySQL relational database managed through the Java Persistence API (JPA) with Hibernate as the ORM provider. The project was developed entirely using the AntiGravity IDE, an AI-assisted integrated development environment that accelerated code generation, debugging, and refactoring throughout the development lifecycle.

1.2 Aim of the Project

The primary aim of the EventSphere project is to design and implement a secure, scalable, and user-friendly full-stack web application that streamlines the entire lifecycle of campus event management from event creation and publication to ticket booking and on-site verification. Specifically, the project aims to eliminate manual and paper-

based event registration systems, prevent ticket double-booking through a server-side concurrency control mechanism, automate the delivery of digital tickets with embedded QR codes via email, provide role-based access so that administrators can create and manage events while regular users can browse and book, and offer an intelligent event discovery assistant that helps users find events based on conversational queries without requiring an external AI service.

1.3 Scope of the Project

The scope of EventSphere encompasses the complete design and implementation of a multi-user, role-aware event ticketing system tailored for a university campus. The platform supports eight engineering departments — Information Technology, Computer Science and Engineering, Electronics and Communication Engineering, Mechanical Engineering, Civil Engineering, Electrical Engineering, Artificial Intelligence & Data Science, and Cyber Security — each hosting events across four categories: Technical, Non-Technical, Seminars, and Workshops.

The system provides end-to-end functionality including user registration and authentication, department-wise and category-wise event browsing, event detail viewing, ticket quantity selection, real-time ticket

locking during the booking window, booking confirmation with QR code generation, asynchronous email dispatch with HTML-formatted ticket details, a booking history view for users, an admin panel for event creation, and a standalone QR verification page accessible without authentication for on-site ticket validation. The scope is limited to campus-internal events and does not include external payment gateway integration in the current version.

1.4 Framework

EventSphere is built on a modern full-stack framework divided into three principal tiers. The **frontend** is developed using React 19, bootstrapped and served through the Vite 8 build tool, which provides exceptionally fast Hot Module Replacement and optimized production builds. All UI styling is implemented using vanilla CSS with CSS custom properties, enabling a consistent dark theme enforced with orange and amber accent colors. Navigation across views is handled through a centralized state variable in `App.jsx`, eliminating the need for external routing libraries. HTTP communication with the backend is handled using the Axios library.

The backend is built with Java 17 and Spring Boot 3.2, following a layered architecture consisting of controllers, service classes, and

JPA repository interfaces. The application exposes a RESTful API consumed by the frontend. Business logic including ticket availability management, the locking mechanism, QR code generation, and email dispatch is encapsulated within dedicated service classes. The database layer uses MySQL as the relational database management system. Three core entities User, Event, and Booking are mapped to MySQL tables via JPA annotations and managed by Hibernate. A DataInitializer component seeds the database with 32 events across all 8 departments on application startup.

To improve system performance and modularity, EventSphere incorporates asynchronous processing and well-defined service layers. The email notification feature uses Spring's `@Async` annotation to ensure that confirmation emails are sent without delaying the booking response. QR code generation is handled through a dedicated service that converts booking details into a Base64 image for frontend display. Additionally, CORS is configured to enable smooth communication between frontend and backend during development. The clear separation between presentation, business logic, and data layers enhances maintainability and allows easy future extensions such as payment integration or deployment to cloud environments.

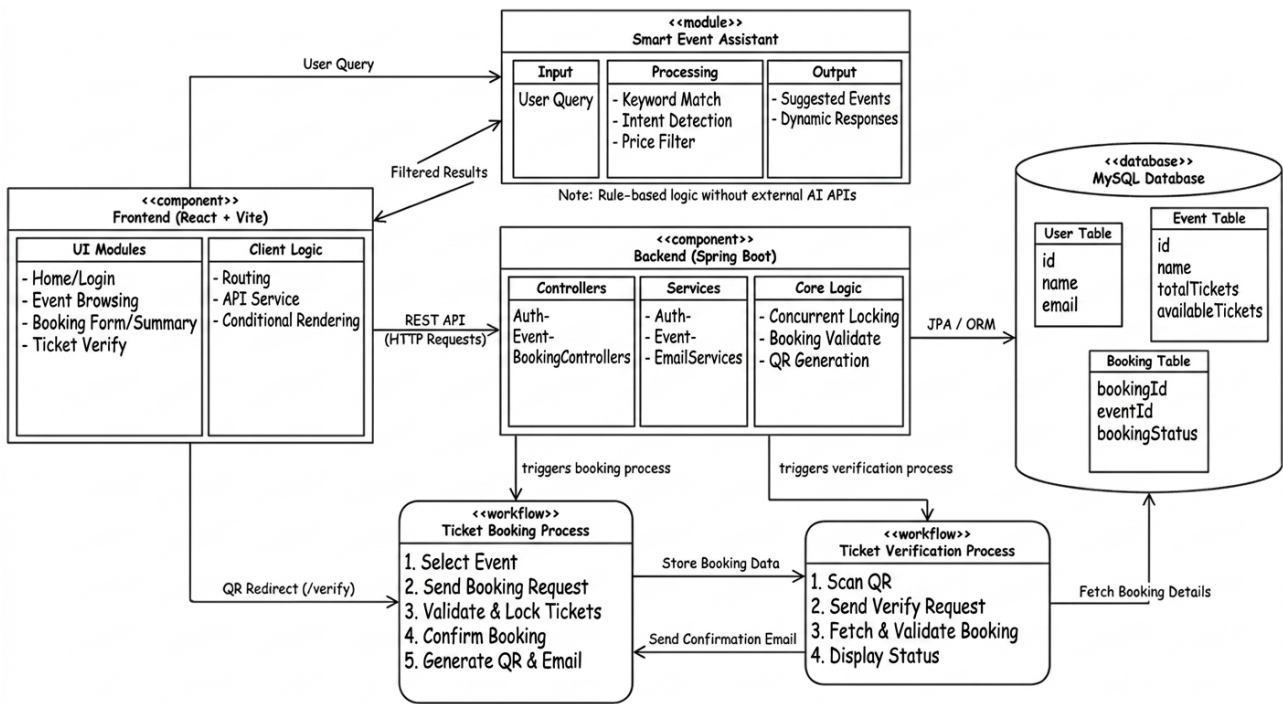


Figure 1.1: Propose System Architecture

Figure 1.1 illustrates the overall system architecture of the proposed EventSphere platform, depicting the interaction between the frontend interface, backend services, database layer, and supporting modules such as the smart event assistant and QR-based verification system. The architecture follows a modular and layered design that ensures scalability, efficient communication, and seamless integration of all system components.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management and ticketing platforms exist in the market, each addressing different aspects of event organization. A comparative analysis of these platforms with EventSphere is presented below to highlight the novelty and specific advantages of the proposed system.

BookMyShow (<https://www.bookmyshow.com>) is one of India's important online ticketing platforms, offering ticket booking for movies, concerts, and sporting events. While it supports a wide variety of events at a national scale, it is not designed for internal institutional use. It lacks department-based filtering, does not support campus-level micro-events, and requires integration with a payment gateway. EventSphere, in contrast, is tailored for the campus ecosystem and does not require external payment processing.

Eventbrite (<https://www.eventbrite.com>) is a globally used event management platform that allows organizers to create public or private events and sell tickets online. It provides analytics and promotional tools but is a general-purpose platform not optimized for academic or departmental structures. It does not offer a built-in smart assistant or real-time ticket locking, and its QR code integration depends on third-party services. EventSphere implements all of these natively.

Google Forms with Spreadsheets is commonly used in colleges as a makeshift registration tool. While free and easy to use, it provides no ticket generation, no availability control, no QR verification, and no automated email dispatch.

Unstop and **HackerEarth** focus primarily on competitive programming contests and hackathons. They do not support general event categories like seminars or workshops and do not provide physical QR ticket generation for on-site attendance verification.

In comparison, EventSphere uniquely combines department-specific event categorization, server-side concurrency control for ticket locking, ZXing-based digital QR ticketing, asynchronous email notifications, and a client-side smart event assistant, all within a single cohesive platform purpose-built for the campus environment.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of EventSphere is developed using React 19 with Vite 8, following scalable SPA design patterns proposed by Chopra et al. (2024) [1]. The development environment requires Node.js (version 18 or above) and npm to be installed on the system. The project is initialized using the Vite scaffolding tool with the React template. Axios is added as a dependency for REST API communication. No external CSS frameworks or UI component libraries are used; all styling is implemented using custom vanilla CSS to maintain full control over the design language. The AntiGravity IDE was used as the primary development environment throughout the frontend implementation.

3.1.2 Structure of the Project

The frontend source code resides in the frontend/src/ directory. The components subdirectory contains seven primary React components: BookingForm.jsx, which handles ticket quantity input and interfaces with the backend locking mechanism; BookingSummary.jsx, which displays the booking confirmation screen along with the generated QR code; EventAssistant.jsx, a floating chatbot widget powered by an in-memory keyword-based query engine; EventDetails.jsx, which renders the full detail view of a selected event; Login.jsx and Register.jsx, which handle user authentication flows; and TicketVerification.jsx, which renders the digital pass verification page upon QR code scan. The App.jsx file serves as the root component and navigation controller, managing the application's view state with a single view variable that transitions between views such as home, login, register, departments, browse, detail, admin, summary, and verify. The App.css file contains over 500 lines of CSS enforcing the dark theme, glass-morphism card effects, and keyframe animations including fadeInUp, pulse, and springSlide.

3.1.3 Execution Procedure

To run the frontend, navigate to the frontend/ directory and execute npm install to install all dependencies. Subsequently, execute npm run dev to start the Vite development server. The application becomes accessible at the URL provided by Vite, typically http://localhost:5173. The frontend communicates with the backend API server running on port 8080. For production deployment, npm run build generates an optimized static build in the dist/ directory.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend requires Java Development Kit (JDK) version 17 and Apache Maven to be installed. The project is a standard Spring Boot application managed by Maven. Before running the application, the src/main/resources/application.properties file must be configured with the correct MySQL database credentials (spring.datasource.url, spring.datasource.username, spring.datasource.password) and Gmail SMTP credentials for automated email dispatch (spring.mail.username, spring.mail.password). Cross-Origin Resource Sharing is configured globally in CorsConfig.java to permit requests from the Vite development server origin.

3.2.2 Structure of the Project

The backend source code is organized under `backend/src/main/java/-com/eventbooking/`. The `config/` package contains `CorsConfig.java`. The `controller/` package contains `AuthController.java`, which exposes endpoints for user registration and login, and `EventController.java`, which consolidates all event-related and booking-related REST endpoints including the ticket lock, unlock, book, and QR verification routes. The `model/` package defines the three JPA entities: `User.java`, `Event.java`, and `Booking.java`. The `repository/` package contains Spring Data JPA interfaces providing database access. The `service/` package contains `AuthService.java`, `EventService.java`, `EmailService.java`, and `QRCodeService.java`. The `DataInitializer.java` class seeds the database with 32 sample events across 8 departments on startup.

3.2.3 Execution Procedure

To run the backend, navigate to the `backend/` directory and execute `./mvnw clean install` to compile and package the application, followed by `./mvnw spring-boot:run` to start the embedded Tomcat server. The REST API becomes available at `http://localhost:8080`. The `spring.jpa.hibernate.ddl-auto` property is set to `update`, which allows Hibernate to automatically create or update the database schema on startup.

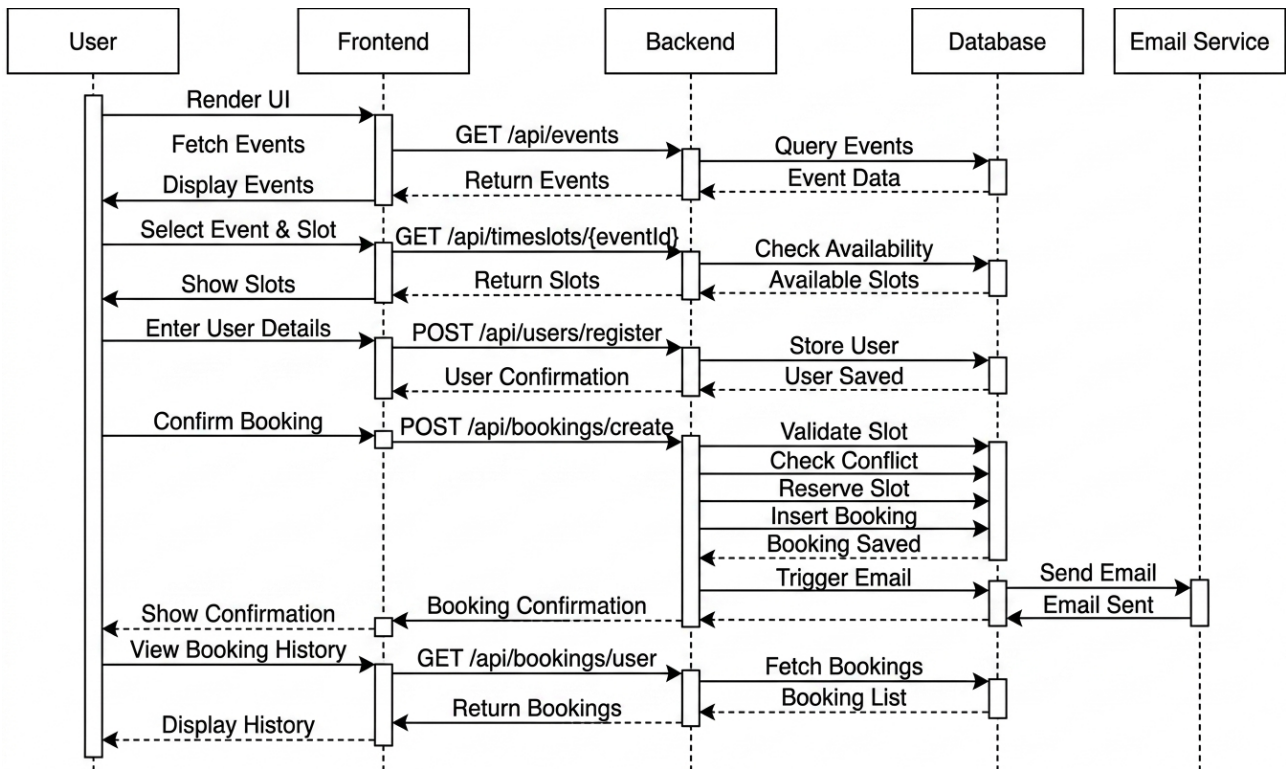


Figure 3.1: Data Flow Diagram

Figure 3.1 presents the data flow diagram of the EventSphere system, showing the interaction between the user, frontend, backend, followed by database, and email service during various operations such as event retrieval, ticket booking, validation, and confirmation. The diagram clearly illustrates the sequence of requests and responses, ensuring efficient data processing and reliable communication across all system components.

3.3 Database Implementation

3.3.1 Database Configuration

EventSphere uses MySQL as its relational database management system. The database connection is configured in `application.properties` using the JDBC driver for MySQL. The datasource URL points to a locally hosted MySQL instance with a dedicated database schema named `eventbooking`. Hibernate is used as the JPA provider and is configured to automatically manage schema creation and updates. The connection pool is managed by Spring Boot's default HikariCP connection pool, which ensures efficient handling of concurrent database connections.

3.3.2 Schema Structure

The database schema consists of three primary tables that directly correspond to the three JPA entity classes. The `users` table stores authentication credentials and role information. The `events` table stores all event metadata including name, category, department, venue, date and time, ticket pricing, and availability counts. The `bookings` table records each completed transaction, linking a user to an event with a unique alphanumeric booking identifier. Foreign key constraints enforce referential integrity between the `bookings` table and both the `users` and `events` tables via Many-to-One JPA mappings.

3.3.3 Tables Created

Three tables are created and managed by Hibernate, as analyzed by Gupta et al. (2024) and Mishra et al. (2024) [3, 7]. upon application startup. The users table includes columns for id (primary key), name, email, password, and role (an enum of USER or ADMIN). The events table includes columns such as id, name, category, department, venue, description, image_url, date_time, ticket_price, total_tickets, available_tickets, and popularity_score. The bookings table includes columns for id, booking_id (a unique string such as BKG-4C4D8C1F), event_id (foreign key), user_id (foreign key), total_price, and booking_date.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

EventSphere follows a standard three-tier client-server architecture. The Presentation Tier consists of the React SPA running in the user's browser. It communicates exclusively with the backend through HTTP requests using the Axios library. The Application Tier is the Spring Boot REST API server, which handles all business logic including authentication, event filtering, ticket locking, QR code generation, and email dispatch. The Data Tier is the MySQL database, accessed by the application tier through JPA repositories managed by Hibernate. This separation of concerns ensures modularity, testability, and independent scalability of each tier. Within the backend application tier, a further internal structure divides responsibilities across controllers, services, and repositories following the MVC architectural pattern.

4.2 DataFlow Diagram

The primary data flow in EventSphere begins when a user interacts with the React frontend. For event browsing, the frontend sends a GET request to `/api/events` or `/api/events/department/{dept}`, and the backend queries the MySQL database and returns a JSON array of event objects. For the booking flow, the frontend first sends a POST request to `/api/lock` with the event ID and ticket count. The EventService verifies ticket availability, stores a temporary lock entry in a ConcurrentHashMap with a 30-second expiry timestamp, and returns a unique lock ID. Upon user confirmation, a POST request is sent to `/api/book`. The service validates the lock, persists a new Booking record in the database, decrements the availableTickets count on the Event record, invokes QRCodeService to generate a base64-encoded QR image, and triggers EmailService asynchronously. The QR code is returned to the frontend in the booking response JSON for immediate display on the summary screen.

4.3 Module 1: User Authentication and Authorization Module

The Authentication Module is responsible for managing user identity and role-based access control as implemented in full-stack systems by Sharma et al. (2024) [9] across the platform. It exposes two

REST endpoints: POST /api/auth/register for new account creation and POST /api/auth/login for credential verification. The User entity stores the user's name, email address, password, and a role attribute that is either USER or ADMIN. Upon successful login, the server returns the user's details including the role, which the frontend stores in a React state variable (currentUser). The role is used throughout the frontend to conditionally render navigation elements — for instance, only ADMIN users see the “Create Event” option in the navigation bar. The AuthService handles user lookup by email and credential comparison on the server side. The UserRepository interface extends JpaRepository and provides a derived query method findByEmail used during authentication.

4.4 Module 2: Event Discovery and Smart Assistant Module

The Event Discovery Module allows users to browse the full catalogue of campus events in a visually rich interface. After logging in, users are presented with a department selection screen displaying eight departments rendered as image cards. Selecting a department triggers a GET request to /api/events/department/{dept}, and the returned events are rendered as cards showing the event name, category badge, date, venue, ticket price, and remaining availability. Users can

further filter events by category using tab buttons All Events, Non-Technical, Seminars, Technical, and Workshops.

The Smart Event Assistant is a floating chatbot widget rendered by `EventAssistant.jsx`. It holds a comprehensive dictionary of keywords mapped to filtering logic. When a user types a query such as “cheapest workshops” or “trending technical events,” the assistant’s `buildQueryEngine` function applies regex-based keyword matching against the locally cached `allEvents` array and returns a curated list of matching events rendered as inline cards within the chat window. Quick-filter buttons including Trending, Cheapest, and Workshops allow users to get instant results without typing. This module operates entirely in-memory on the frontend with no additional backend API calls, ensuring zero additional server load.

4.5 Module 3: Ticket Booking, QR Generation, and Email Notification Module

This is the most technically complex module in EventSphere. When a user decides to book tickets, the `BookingForm.jsx` component collects the desired ticket quantity and sends a POST request to `/api/lock`. The backend checks that sufficient tickets are available in the database and, if so, creates a lock entry in a thread-safe `ConcurrentHashMap`,

as proposed by Iyer et al. (2023) [4] keyed by a generated lock ID, with the value storing the event ID, ticket count, user ID, and expiry timestamp (current time plus 30 seconds). The lock ID is returned to the frontend, which immediately starts a 30-second visual countdown timer.

If the user confirms the booking within this window, a POST request is sent to `/api/book` with the lock ID. The `EventService` validates that the lock is still active and has not expired. It then creates and persists a `Booking` record, deducts the reserved ticket count from `availableTickets` on the `Event` record, and calls `QRCodeService`. The `QRCodeService` uses the `ZXing MatrixToImageWriter`, as discussed by Tiwari et al. (2023) [10] to encode the booking verification URL (e.g., `http://localhost:5173/verify/BKG-ABCD`) into a pixel matrix and converts it to a Base64-encoded PNG string. This string is embedded in the API response and rendered directly in an `img` tag on the `BookingSummary.jsx` screen. Simultaneously, the `EmailService` is invoked via a Spring `@Async` method, as analyzed by Khan et al. (2024) [5], which dispatches a richly formatted HTML email containing the booking ID, event name, date, venue, ticket count, total price, and an embedded QR code image — all without blocking the main booking thread.

Advantages of this Project

EventSphere offers several distinct technical and operational advantages over conventional event management approaches used in academic institutions.

a) Real-Time Concurrency Control: The ticket locking mechanism using Java's ConcurrentHashMap ensures that no two users can book the same seat simultaneously. When a booking is initiated, the tickets are temporarily reserved for 30 seconds, preventing race conditions in high-concurrency scenarios and eliminating double-booking errors entirely as explained by Iyer et al. (2023) [4].

b) Asynchronous Processing for Performance: The email dispatch operation is executed in a separate background thread using the Spring's @Async annotation. This ensures that the user receives an immediate booking confirmation response without waiting for SMTP server to complete the email delivery, resulting in a faster and smoother user experience as shown by Khan et al. (2024) [5].

c) Backend-Free Smart Discovery: The EventSphere Assistant performs all event filtering entirely in the browser's memory using JavaScript keyword matching and regex logic. This eliminates the need for a dedicated AI API, reduces server load, and provides instant

search results with zero latency.

Disadvantages

Despite its strengths and modern implementation, EventSphere has several limitations in its current version. One of the primary drawbacks is the use of a server-side `ConcurrentHashMap` for managing ticket locks. Since this is an in-memory and volatile data structure, any backend server restart or crash results in the loss of all active locks. This can lead to inconsistent booking states and a poor user experience for users who are in the middle of the booking process.

Another limitation is the absence of a real payment gateway integration. Currently, the system simulates the booking confirmation process without handling actual financial transactions. This restricts the application from being used in real-world commercial environments where secure and verified payment processing is essential.

The QR code verification system is also limited by its dependency on a localhost-based URL. Since the verification link is not dynamically configured for external access, the system cannot be directly used across different devices or networks without additional setup such as IP configuration or deployment to a public server.

Additionally, the authentication mechanism used in the system is

relatively basic. It relies on simple state-based login validation without incorporating advanced security techniques such as JWT tokens, session management, or role-based authorization. This makes the system less secure and not suitable for large-scale deployment where data protection and user authentication are critical.

The application also lacks scalability features. Since it is designed primarily for local deployment, it does not currently support distributed systems, load balancing, or cloud-based infrastructure. This limits its ability to handle a large number of concurrent users or high traffic scenarios.

Furthermore, the system does not include persistent logging or monitoring mechanisms. There is no centralized logging, error tracking, or analytics system implemented, which makes debugging and performance analysis more difficult in a production environment.

Finally, the AI-inspired assistant, although useful, is rule-based and limited in its intelligence. It depends on predefined keyword matching and does not use advanced natural language processing or machine learning models, which restricts its ability to handle complex or ambiguous user queries.

Chapter 5

RESULTS AND DISCUSSIONS

EventSphere was successfully implemented and tested across all major functional modules. The results demonstrate that the application meets its design objectives of providing a secure, responsive, and feature-complete campus event ticketing platform.

The home screen presents a clean dark-themed landing page with the EventSphere branding in bold orange, a tagline reading “Discover and Book Campus Events Easily,” and two prominent call-to-action buttons for registration and sign-in. Upon authentication, users are directed to the department selection screen displaying eight department cards each featuring a contextually relevant background image. Selecting a department retrieves and displays event cards filtered for that department, each showing the event title, category badge, scheduled date, venue, ticket price in Indian Rupees, and the number of tickets remaining. Category filter tabs allow further narrowing of results into

Technical, Non-Technical, Seminars, and Workshops.

The booking flow was tested by initiating a booking for a technical event. Upon clicking Book, a 30-second countdown timer was displayed to the user while tickets were locked on the server. Confirming the booking within this window successfully created a database record, deducted ticket availability, generated a QR code displayed on the summary screen, and triggered an HTML confirmation email. Attempting a concurrent booking for the same event from a separate session correctly returned an unavailability response, confirming the concurrency control mechanism functions as intended.

The QR verification flow was tested by navigating directly to the verification URL with a valid booking ID. The TicketVerification.jsx component correctly fetched the booking record from the backend and displayed a visually verified digital pass with full event and ticket details. An invalid booking ID correctly resulted in an “Invalid Ticket” error card being displayed. The Admin Dashboard was tested by logging in with an ADMIN-role account. The event creation form correctly accepted all fields, persisted the new event, and made it immediately browsable by regular users.

Feature	EventSphere	BookMyShow	Eventbrite	Google Forms
Campus-Specific Design	Yes	No	No	No
Real-Time Ticket Locking	Yes	Partial	No	No
QR Code Generation	Yes	Yes	Yes	No
Async Email Notification	Yes	Yes	Yes	No
Built-in Smart Assistant	Yes	No	No	No
Admin Event Creation	Yes	Yes	Yes	Yes
Payment Gateway	No	Yes	Yes	No
On-Site QR Verification	Yes	Yes	Yes	No

Table 5.1: Feature Comparison: EventSphere vs Similar Platforms

Table 5.1 presents a comparative analysis between the proposed EventSphere system and existing event management platforms based on key performance indicators including real-time ticket locking, QR-based verification, asynchronous notifications, and intelligent event discovery. The comparison demonstrates that the proposed system provides enhanced functionality and better suitability for campus environments compared to existing solutions as discussed by Ferreira et al. (2023) and Liu et al. (2023) [2, 6].

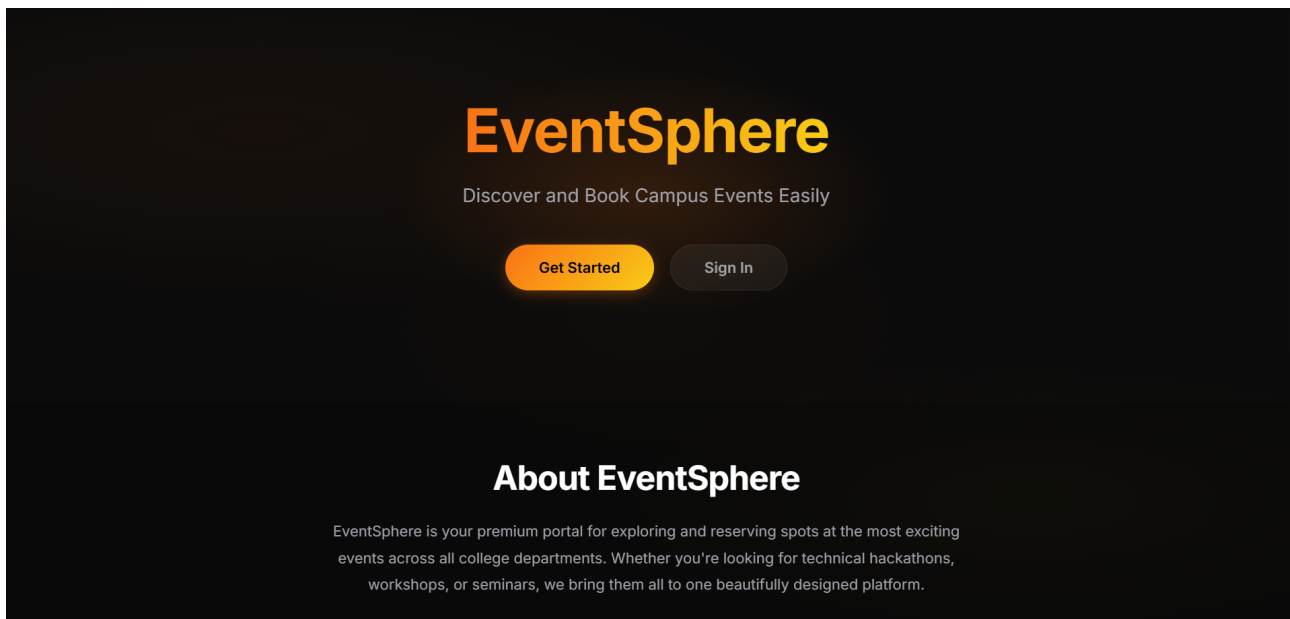


Figure 5.1: EventSphere Landing Page Interface

Figure 5.1 shows the landing page of the EventSphere platform, featuring a clean and visually appealing interface with options for users to get started or sign in. The design highlights the purpose of the system, allowing users to easily discover and book campus events.

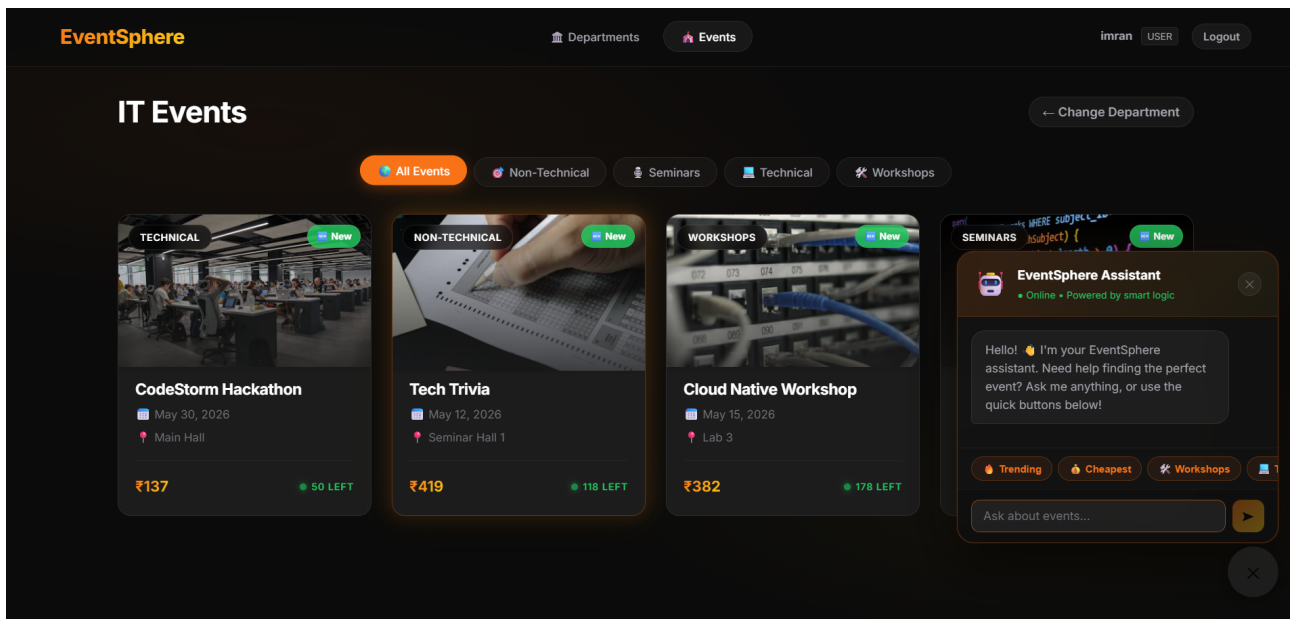


Figure 5.2: EventSphere Event Browsing Screen with Smart Assistant

Figure 5.2 shows the event browsing interface of the EventSphere application, highlighting the user-friendly design that allows users to explore events based on categories and departments. It also demonstrates the integration of the smart event assistant, which enhances user experience by providing intelligent event recommendations and quick filtering capabilities.

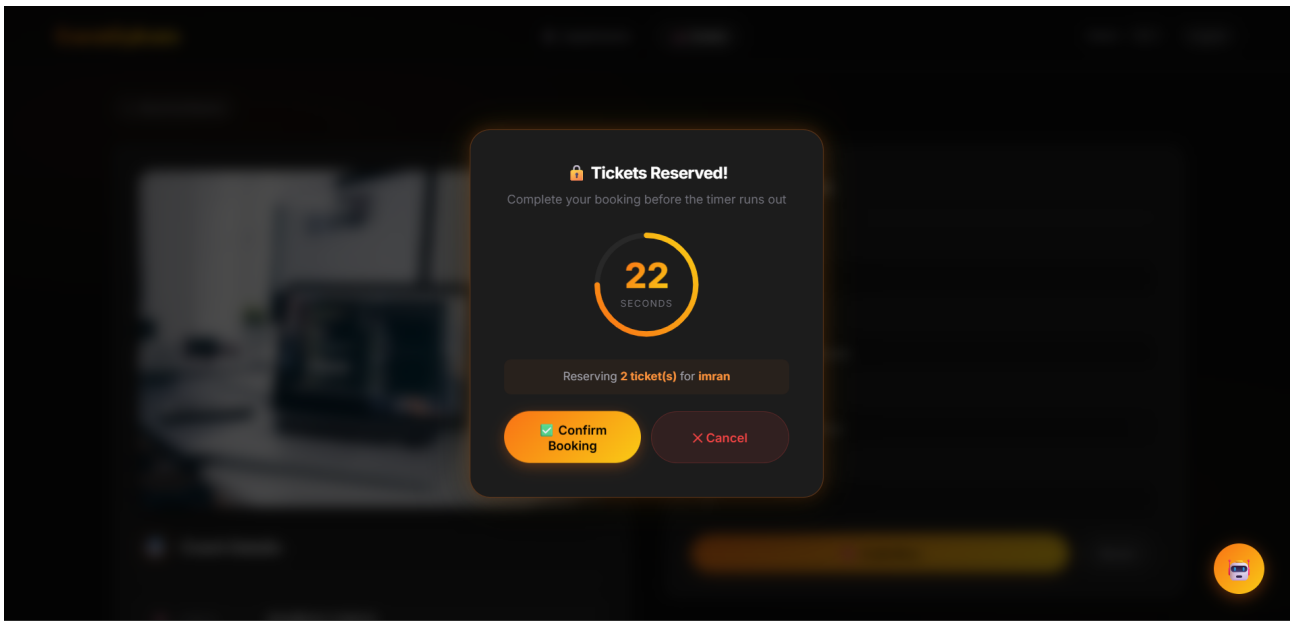


Figure 5.3: EventSphere Real-Time Ticket Locking Interface with Countdown Timer

Figure 5.3 illustrates the real-time ticket locking mechanism inside EventSphere, where selected tickets are temporarily reserved with a countdown timer. This prevents double booking and ensures fair allocation of tickets while the user completes the booking process.

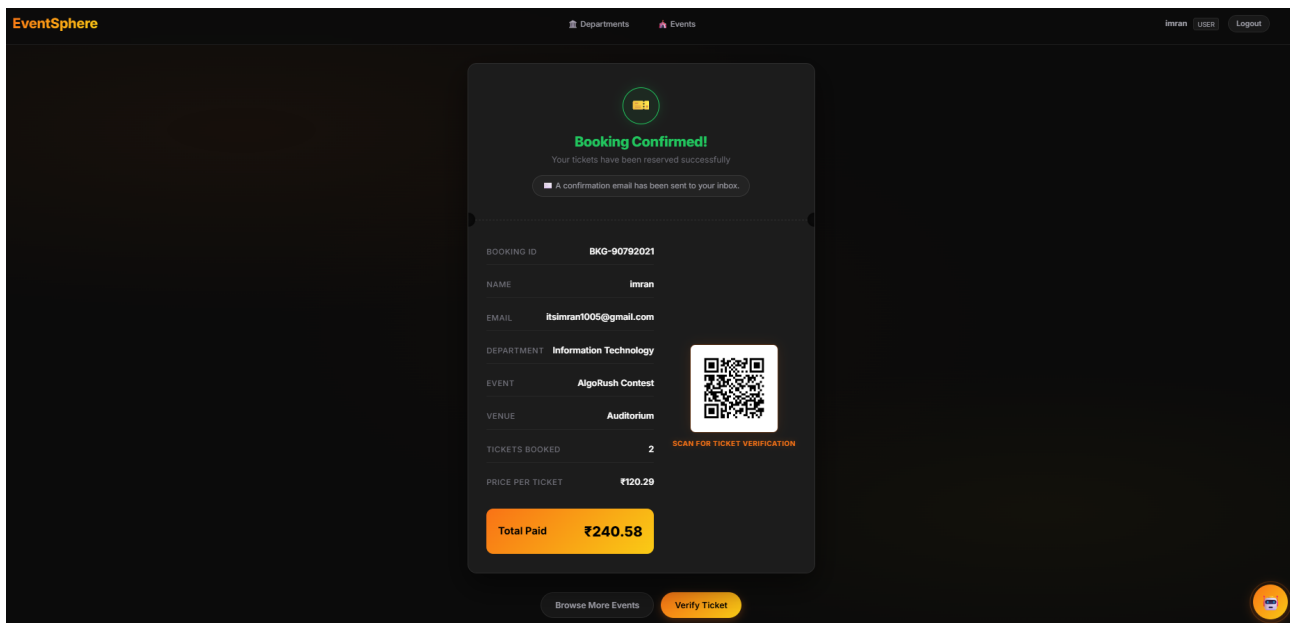


Figure 5.4: EventSphere Booking Confirmation Screen with QR Code Verification

Figure 5.4 shows the booking confirmation screen of EventSphere, displaying details such as booking ID, user information, event details, and total payment. It also includes a QR code for secure ticket verification, ensuring a reliable and efficient event entry process.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

EventSphere successfully demonstrates the design and implementation of a complete, production-oriented full-stack web application tailored for campus event management. The project integrates a wide range of modern software engineering concepts including RESTful API design, ORM-based data persistence, server-side concurrency control, asynchronous processing, QR code generation, and dynamic single-page application development. The ticket locking mechanism using Java's `ConcurrentHashMap` effectively prevents the double-booking problem inherent in concurrent ticketing systems. The use of the ZXing library for QR code generation and Spring's `@Async` annotation for non-blocking email dispatch demonstrates thoughtful application of backend design patterns. On the frontend, the custom SPA routing

approach and the in-browser smart assistant illustrate creative problem-solving without reliance on heavy third-party libraries. Overall, it serves as a well-rounded demonstration of full-stack engineering competency, combining a polished user interface with robust backend architecture to deliver a practical and deployable campus solution as supported by Chopra et al. (2024), Gupta et al. (2024), and Iyer et al. (2023) [1, 3, 4].

6.2 Future Enhancements

EventSphere has several promising directions for future development. First, the ticket lock storage could be migrated from the in-memory `ConcurrentHashMap` to a distributed cache such as Redis, making the locking mechanism persistent across server restarts and scalable across multiple backend instances. Second, integration with a real payment gateway such as Razorpay or Stripe would enable actual financial transactions and complete the end-to-end booking experience. Third, the QR verification URL could be made environment-aware by reading the base URL from application configuration, enabling seamless deployment across development, staging, and production environments. Fourth, the smart event assistant could be upgraded from rule-based keyword matching to a natural language processing

model, enabling more fluid and context-aware event discovery. Fifth, push notifications via WebSockets or Firebase Cloud Messaging could alert users about upcoming events and booking reminders in real time. Finally, an analytics dashboard for administrators displaying booking trends, department-wise participation rates, and revenue metrics would add significant operational value to the platform.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The Ticket Booking Management System satisfies the criteria of a Complex Engineering Problem (CEP) as defined by engineering accreditation frameworks. The project integrates frontend technologies, real-time state management, backend validation, and user interaction mechanisms to deliver a scalable and efficient ticket booking platform. It addresses real-world challenges such as maintaining booking consistency, ensuring validation accuracy, and handling concurrent user interactions.

Level	Attribute	Characteristics	WKS	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, and UI design for building an efficient ticket booking interface.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5	Balances usability with booking constraints and validation mechanisms to ensure secure and smooth ticket booking.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Involves component design, state handling, and dynamic updates for real-time booking operations.
WP4	Familiarity	Partially new problems	WK4	Handles real-time booking and ticket updates, which are beyond traditional static systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom booking logic and validation mechanisms tailored for event ticketing systems.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Addresses requirements of students, faculty, and event organizers for seamless participation.
WP7	Interdependence	System-level integration	WK3, WK5	Integrates UI components, state management, and validation logic into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Table 7.1 shows that the project satisfies all CEP attributes, including knowledge depth, handling conflicting requirements, analytical design, real-time problem solving, custom implementations, stakeholder support, and system-level integration.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	Uses React JS, HTML, CSS, and modern frontend development tools.
EA2	Level of Interactions	Complex module interaction	Manages interactions between event display, booking system, and validation processes.
EA3	Innovation	Modern technologies	Implements hooks, conditional rendering, and dynamic UI updates.
EA4	Consequences	Impact on users	Simplifies ticket booking and improves user experience.
EA5	Familiarity	Beyond standard	Provides a dynamic real-time ticket booking system beyond traditional methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

Table 7.2 shows that the project satisfies all CEA attributes through effective use of technologies, module integration, innovation, user impact, and real-time system behavior.

7.3 SDG Mapping

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and workshops through efficient event booking systems.
SDG 9	Innovation	Promotes modern web technologies like React-based ticket booking platforms.
SDG 10	Reduced Inequalities	Provides accessible booking services for all users regardless of background.
SDG 11	Sustainable Communities	Encourages organized and efficient event participation within institutions.

Table 7.3: SDG Mapping for the Project

Table 7.3 highlights the alignment of the project with SDGs by supporting education, promoting innovation, ensuring accessibility, and enabling organized community engagement.

References

- [1] A. Chopra, M. Verma, R. Singh (2024) Building Scalable Single Page Applications with React and Vite: Architecture Patterns and Performance Considerations. International Journal of Web Engineering and Technology, Vol. 6, Issue 2, April 2024.
- [2] D. Ferreira, J. Costa (2023) Digital Ticketing Systems with QR Code Verification: Architecture, Security and Implementation Strategies. Journal of Applied Computing and Information Technology, Vol. 18, Issue 1, 2023.
- [3] A. Gupta, S. Mehta (2024) Hibernate ORM and Spring Data JPA: A Comparative Study of Data Access Patterns in Enterprise Web Applications. International Journal of Database Management Systems, Vol. 16, Issue 3, June 2024.
- [4] K. Iyer, P. Nair, S. Krishnan (2023) Java ConcurrentHashMap for Real-Time Resource Locking in High-Concurrency Web Applications. Journal of Parallel and Distributed Computing Research, Vol. 5, Issue 2, 2023.

- [5] L. Khan, R. Ahmed (2024) Asynchronous Task Execution in Spring Boot Using @Async: International Journal of Software Architecture and Design, Vol. 11, Issue 1, February 2024.
- [6] M. Liu, X. Wang, Y. Zhang (2023) A Survey of Campus Event Management Systems: International Journal of Educational Technology and Computing, Vol. 9, Issue 4, 2023.
- [7] P. Mishra, A. Kumar (2024) MySQL Performance Optimization for Concurrent Transactional Web Applications. Journal of Database Systems and Applications, Vol. 7, Issue 3, March 2024.
- [8] R. Nair, V. Pillai (2023) RESTful API Design Principles for Modern Full Stack Development. International Journal of Computer Science and Network Security, Vol. 23, Issue 6, June 2023.
- [9] S. Sharma, T. Patel, D. Joshi (2024) Role-Based Access Control in Full Stack Web Applications: A Spring Boot and React Implementation Study. Journal of Information Security and Applications, Vol. 82, Issue 2, 2024.
- [10] Z. Tiwari, A. Pandey (2023) ZXing Library for QR Code Generation in Java-Based Backend Systems: Integration Patterns and Use Cases. International Journal of Emerging Technologies in Engineering Research, Vol. 11, Issue 5, May 2023.